

HaReCAP: Habitual-action Grounding for Recursive Large Language Model Agents

Shen Liu¹, Zhenguo Xu², Shaopu Wang³, Yike Gao³, and Chunlei Wang^{3*}

¹ North China Institute of Computer System Engineering, Beijing, China
liushen24@mails.ucas.edu.cn

² University of Science and Technology of China, Hefei, China
xzg@mail.ustc.edu.cn

³ China Information Security Research Institute Co., Ltd., Beijing, China
wangshaopu18@mails.ucas.ac.cn, yikegao@mail.ustc.edu.cn,
wangchl@cecr.ac.cn

Abstract. Long-horizon embodied tasks require LLM agents to iteratively decompose high-level goals, revise plans in response to environmental feedback, and ground leaf-level subgoals into valid executable actions. Recursive context-management methods such as ReCAP improve planning stability through multi-level task decomposition and parent-node refinement, but still repeatedly invoke the LLM at leaf nodes to ground atomic subtasks into exact valid actions. We refer to this final grounding step as *last-mile grounding redundancy*, which accumulates into substantial LLM-call and token overhead during long-horizon execution. To mitigate this issue, we propose HaReCAP⁴ (Habitual-action Grounded ReCAP), a low-intrusion leaf grounding extension for ReCAP. HaReCAP extracts frequent leaf decisions from successful trajectories and compiles them offline into auditable and abstainable one-step leaf-reflex rules. At runtime, it skips the leaf LLM call only when a rule can uniquely determine a legal action in the current valid-action set; otherwise, it falls back to the original ReCAP. This design avoids repeatedly carrying the full recursive context into the LLM for routine leaf action grounding, while preserving the original recursive control flow. We evaluate HaReCAP on Robotouille and ALFWorld with Qwen3.5-72B as the main model. On tasks solved by both ReCAP and HaReCAP, HaReCAP reduces token consumption by 14.67%, 17.93%, and 20.08% on Robotouille synchronous, Robotouille asynchronous, and ALFWorld, respectively. The results show that HaReCAP can serve as a low-intrusion extension to ReCAP-style recursive context-management frameworks, reducing *last-mile grounding redundancy* across environments and models on commonly successful trajectories.

Keywords: large language model agents · recursive planning · leaf-level reasoning distillation · habitual-action grounding

* Corresponding author.

⁴ Code will be released upon publication to facilitate reproducibility.

1 Introduction

Long-horizon embodied tasks require large language model (LLM) agents to perceive the environment, decompose goals, update plans, and produce executable actions over continuous interaction. Unlike one-shot question answering or short-form reasoning, these tasks expose the model to changing observations, local subgoals, and valid-action constraints. Consequently, the agent must maintain the consistency of the goal and the legality of the action in a dynamic context. Chain-of-Thought (CoT) and Tree-of-Thoughts (ToT) highlight the importance of explicit reasoning processes and search-based reasoning strategies for improving complex task-solving capabilities [5, 6]. Extending this line of work, ReAct couples reasoning with action execution, allowing models to iteratively update their decisions through environmental feedback, thereby improving performance in interactive decision-making tasks [4]. Recently, ReCAP [1], a state-of-the-art framework for long-horizon execution, introduces recursive context management to support complex task completion. It organizes execution through plan-ahead decomposition, parent-plan re-injection, and memory-efficient execution: the model generates a subtask list at the current node, the system recursively descends into the first subtask, and after that subtask is completed, the remaining parent plan is re-injected and refined with the latest observation. This mechanism avoids naive linear accumulation of the full interaction history and has shown clear advantages on long-horizon benchmarks such as Robotouille [2].

However, ReCAP[1] does not remove all repeated reasoning. At each leaf node, the model still has to translate a natural-language subtask into an action string that exactly matches the current valid-action set. For example, once recursive decomposition has produced the leaf subtask “pick up onion1 from board1”, the leaf LLM call often only grounds it into the exact valid action “Pick up onion1 from board1 using robot1”. Although this step appears simple, it still carries the ReCAP[1] dialogue history, parent-task context, latest observation, and output JSON schema, consuming another round of prompt and completion tokens. This leaf-level action-grounding overhead is repeatedly amplified, since long-horizon tasks contain many atomic actions. We call this repeated reasoning process at the end of recursive planning *last-mile grounding redundancy*. As shown in Fig. 1, among successful Qwen3.5-27B ReCAP[1] trajectories, strict leaf/action grounding calls account for 23.67%, 29.56%, and 45.73% of all LLM calls on Robotouille[2] synchronous, Robotouille[2] asynchronous, and ALFWorld[15], respectively. This indicates that last-mile grounding is not an incidental edge cost, but a stable component of inference cost in recursive LLM agents.

A direct alternative is to learn intermediate macro skills, such as compiling “move onion from fryer to board” into a sequence of pick-up, move, and place actions. However, in a recursive framework such as ReCAP[1], mid-level macro skills introduce additional risks: they must resolve object bindings, estimate subgoal progress, and decide skill termination. Direction-sensitive actions such as “Stack A on B” versus “Stack B on A” can fail when object roles are reversed; under layout changes, a mid-level macro skill may also execute before

Benchmark	ReCAP call composition	Grounding
Robotouille Sync		23.67%
Robotouille Async		29.56%
ALFWorld		45.73%

■ recursive reasoning/refinement ■ strict leaf/action grounding

Fig. 1. Last-mile action-grounding redundancy in successful ReCAP trajectories. Orange denotes strict leaf/action grounding calls, while blue denotes the remaining recursive reasoning, decomposition, and refinement calls. Statistics are collected from the Qwen3.5-72B ReCAP baseline.

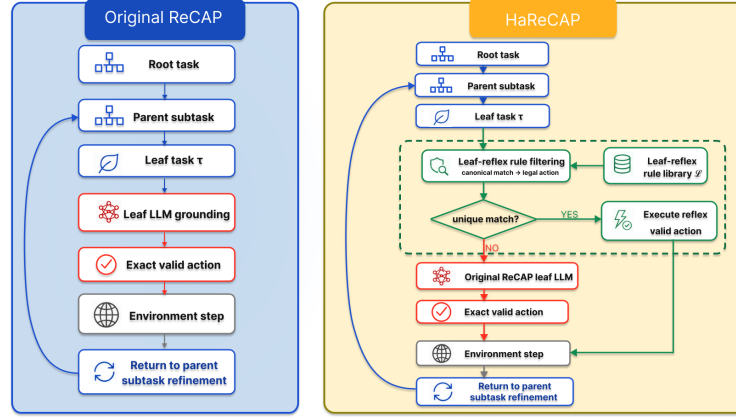


Fig. 2. Leaf-level difference between original ReCAP and HaReCAP. HaReCAP preserves recursive planning, backtracking, and refinement, and only inserts a leaf-reflex library at leaf-level action grounding.

its preconditions are satisfied. These issues can alter ReCAP[1]’s stable recursive decomposition, backtracking refinement, and failure-handling logic.

To avoid these risks, this paper adopts a more focused and more predictable design: distilling only the final action-grounding step at leaf nodes. We propose HaReCAP, Habitual-action Grounded ReCAP. As summarized in Fig. 2, HaReCAP preserves ReCAP[1]’s recursive task tree and high-level planning, and inserts a local, abstainable, one-step habitual-action grounding module at leaf-level action grounding. To avoid terminology ambiguity, we use *macro skill* to refer to a multi-step process at an intermediate node, such as a subroutine that continuously executes pick-up, move, and place;

A *leaf-reflex rule* in HaReCAP denotes a one-step mapping from a canonical leaf task to a canonical action template. All leaf-reflex rules form a *leaf-reflex library*, which is built offline from successful ReCAP[1] trajectories and is not updated online. *Habitual-action grounding* refers to the runtime process of retrieving rules from the leaf-reflex library and uniquely instantiating a legal action from the current valid-action set. The system first extracts leaf tasks,

current valid actions, and executed actions from successful ReCAP[1] baseline trajectories, and compiles them offline into the leaf-reflex library. During online execution, if the current leaf task can be uniquely grounded by a library rule and the current valid actions, the system directly executes the action; otherwise, it fully falls back to ReCAP[1].

The core claim of this design is: *habitual actions need not replace recursive reasoning; moving frequent leaf-level action grounding out of LLM calls can reduce token cost while preserving the reliability of ReCAP’s task decomposition.* HaReCAP is neither a macro-skill planner nor an online episodic memory system. It does not change ReCAP[1]’s recursive decomposition, backtracking refinement, or failure-handling logic. It is closer to last-mile reasoning distillation: ReCAP[1]’s repeatedly successful final-step action choices are distilled into local reflexes that remain strictly constrained by the current valid-action set.

The contributions of this paper are as follows:

- We identify last-mile grounding redundancy in ReCAP-style recursive context-management frameworks, showing that repeated calls from atomic subtasks to exact valid actions constitute an independently optimizable inference cost.
- We propose habitual-action grounding, which distills frequent successful leaf decisions into abstainable one-step leaf-reflex rules, retaining experience reuse while avoiding macro-skill takeover risks.
- We design a leaf-reflex library construction procedure and a conservative triggering mechanism, so a habitual action is executed only when the current subtask can be uniquely grounded to a legal action, yielding a low-intrusion integration aligned with the original ReCAP[1] control flow.
- We evaluate the mechanism on Robotouille[2] and ALFWorld[15], separate end-to-end performance from paired-success efficiency, and show that HaReCAP reduces LLM calls and token consumption on commonly solved trajectories across environments.

2 Related Work

2.1 Reasoning and Planning for LLM Agents

The core challenge for LLM agents is not merely generating a natural-language plan, but maintaining consistency among observations, local goals, and executable actions. Chain-of-Thought[5], Tree-of-Thoughts[6], and Graph-of-Thoughts[7] improve complex problem solving through explicit reasoning or search structures, but they mainly target static or weakly interactive problems. ReAct[4] interleaves reasoning and acting so the model can adjust the next action based on environment feedback; SayCan[9], Inner Monologue[10], and LLM-Planner[11] further emphasize that language plans must be constrained by affordances, feedback, and executability. In text-based embodied environments such as Robotouille[2], each action must also exactly match the current valid-action set. Thus, even when the model has inferred the correct atomic intent, it still needs to convert that intent into the environment’s action template. This paper focuses precisely on this leaf-level grounding cost.

2.2 Context Management and Structured Memory

Long-horizon interaction creates context growth, stale observations, and goal drift. ReCAP[1] uses a recursive task tree to maintain parent tasks, active subtasks, and backtracking relations, organizing long histories into locally reasoned contexts. External memory and graph structures address context limits from another direction: RAG[16] and GraphRAG[17] organize relevant knowledge as retrievable context, while Plan-on-Graph[18] and Graph-Informed Action Generation[3] explicitly structure states, actions, or historical experience for similar-experience retrieval, loop detection, or limited lookahead planning. These methods typically support long-horizon decision making by expanding retrievable information or reorganizing planning states. HaReCAP enters at a different point: it does not expand ReCAP[1]’s global context structure, nor does it rely on scene graphs, embedding retrieval, or environment simulators. Instead, it compresses repeated leaf-level action grounding decisions from successful ReCAP[1] trajectories into local, verifiable rules. Thus, HaReCAP focuses on action-grounding redundancy at the end of recursive context management rather than replacing ReCAP[1]’s long-horizon memory or planning structure.

2.3 Experience Reuse and Macro Skill Libraries

Long-term memory and skill reuse can reduce repeated exploration. Reflexion[8] summarizes failure experience through verbal feedback; Generative Agents[13], MemGPT[14], and Voyager[12] demonstrate the role of long-term memory or open-ended skill libraries in continual interaction; experience retrieval in embodied tasks is also commonly used to provide similar historical trajectories or guide action selection [22, 3]. However, the larger the skill granularity, the more the system must decide preconditions, object bindings, interruption timing, and parent-goal completion. In environments with stacking order, container state, and asynchronous waiting, mid-level macro skills may bypass ReCAP[1]’s backtracking refinement. We therefore use a more conservative leaf-reflex rule: each rule represents only a one-step mapping from a previously successful leaf task to a valid action. If the current task and valid actions cannot uniquely determine an action, the system abstains and falls back to the original ReCAP[1].

3 Method

3.1 Problem Formulation and Overview

Given a root task g , a current observation o_t , and a valid-action set A_t , ReCAP maintains a recursive task tree $\mathcal{T}$. Each node v corresponds to a natural-language task τ_v , and the model outputs

$$r_v = (\text{think}_v, \text{subtasks}_v). \quad (1)$$

Here r_v denotes the structured response for node v ; think_v is a concise rationale about the current task, environment state, and decomposition decision, while

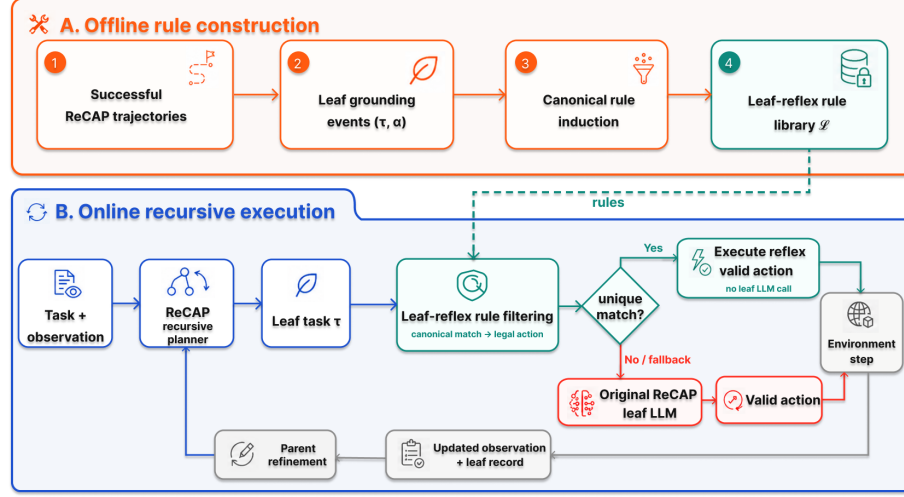


Fig. 3. System framework of HaReCAP. Offline construction builds a leaf-reflex library from successful ReCAP trajectories. Online execution retrieves only before leaf-level action grounding and falls back to the original ReCAP when no rule hits or when ambiguity is detected.

subtasks_v = [τ_{v,1}, ..., τ_{v,n}] is an ordered list of subtasks to be recursively processed. When subtasks_v contains multiple subtasks, ReCAP descends into the first one. When the current subtask has been decomposed to an executable level, the model usually outputs a subtask list containing one candidate action, and the system executes it only if $a \in A_t$. This exact-valid-action constraint guarantees legal interaction with the environment, but it also creates repeated leaf-level grounding cost. Many leaf tasks are already close to action templates, e.g., “Pick up bread1 from table2”, yet the model still needs another call to output “Pick up bread1 from table2 using robot1”.

We define this cost as last-mile action-grounding redundancy in recursive planning and insert a habitual-action grounding module before ReCAP’s leaf LLM call:

$$\pi_H(\tau, A_t; \mathcal{L}) \rightarrow a \in A_t \text{ or } \perp, \quad (2)$$

where τ is the current leaf task, A_t is the valid-action set, $\mathcal{L}$ is the leaf-reflex library, and $\perp$ denotes abstention. If π_H returns a unique action, the system directly executes that valid action; otherwise it fully falls back to the original ReCAP leaf LLM call.

3.2 Offline Construction of Leaf-Reflex Rules

The leaf-reflex library is constructed from successful ReCAP executions. Rather than memorizing complete trajectories or reusable multi-step skills, it retains

only the final action-grounding mappings that have been validated through successful execution. When a leaf task is grounded to a valid executable action, the resulting task-action pair is compressed into a leaf-reflex rule:

$$s = (k_\tau, k_a), \quad (3)$$

where k_τ is the canonical key of the leaf-task text and k_a is the canonical key of the executed action. Intuitively, a rule states that a class of sufficiently concrete leaf tasks should usually be based on a class of action templates; it is not a fixed script for a particular observation.

Rule extraction follows three principles. First, the extraction granularity is restricted to the local mapping between a leaf task and an executed action; parent subtrees, full task trajectories, and multi-action windows are not compiled into the library. This prevents the leaf-reflex library from degenerating into a trajectory replay or a macro skill library. Second, rules are extracted only from successful ReCAP trajectories, ensuring that the library stores one-step grounding decisions already verified by ReCAP rather than unverified intermediate plans. Finally, support is counted after task-level deduplication: repeated occurrences of the same (τ_i, a_i) within a single task count only once, preventing loops inside one trajectory from inflating rule confidence.

To improve reusing across object ids and local layouts, we use action-type-agnostic canonicalization rather than hand-written rules for each action class. This process preserves the word-order structure of tasks and actions, while normalizing object ids into type placeholders, e.g., `bread12` $\rightarrow$ `bread#`. Thus leaf tasks and actions are mapped to

$$k_\tau = \text{canonical_leaf_text}(\tau), \quad (4)$$

$$k_a = \text{canonical_leaf_text}(a). \quad (5)$$

A candidate event enters the library only if k_τ and k_a are non-empty, the leaf task explicitly contains object ids, and those ids are covered by the executed action:

$$\text{ids}(\tau) \neq \emptyset, \quad \text{ids}(\tau) \subseteq \text{ids}(a). \quad (6)$$

This prevents overly abstract leaf tasks such as “prepare ingredient” from being compiled into executable reflexes.

Each leaf-reflex rule is uniquely identified by (k_τ, k_a) and maintains its occurrence count in successful trajectories as support n_s . In the frozen main experiments, the leaf-reflex library is built offline from successful ReCAP trajectories and is not updated online; runtime triggering is therefore mainly controlled by the support threshold and the unique-action constraint. The library size and support-threshold ablation are reported in the experimental section.

3.3 Runtime Retrieval and Triggering

At runtime, HaReCAP executes the task according to ReCAP’s parent-node decomposition, backtracking control, and root-task completion judgment. The

habitual-action grounding module is invoked only immediately before a leaf-level LLM grounding call. Its inputs are restricted to the current leaf task τ , current valid-action set A_t , leaf-reflex library $\mathcal{L}$, and support threshold m . The module does not generate new actions; instead, it checks whether historical rules can uniquely support an already legal action in the current A_t . To avoid triggering on incomplete subgoals, the system first requires τ to have a non-empty canonical key and explicit object ids. It then accepts only rules whose task key matches, whose action key can be uniquely instantiated from current valid actions, whose candidate action covers the object ids in the leaf task, and whose support satisfies $n_s \geq m$. After these filters, each candidate rule s corresponds to at most one valid action in the current environment. Thus, triggering is primarily governed by strict canonical matching and the unique-valid-action constraint. When multiple rules pass the hard constraints and map to executable actions, the system uses a support-derived score $\rho(s)$ as a tie-breaker:

$$\rho(s) = 1.6 \frac{n_s + 1}{n_s + 2} + \frac{1}{10} \min(n_s, 10). \quad (7)$$

The function is monotonic in n_s , so higher-support rules are preferred in conflicts. The first term provides a confidence-like gain that saturates as support increases, while the second term preserves approximately linear separation for $n_s \leq 10$. If two candidates map to different valid actions, the top score must exceed the second score by at least 1; otherwise the candidates are considered within the ambiguity window and the system abstains. This design allows a low-support rule to be overridden by a clearly higher-support rule, but falls back to ReCAP when two candidates are both reasonably supported and cannot be clearly separated. This conflict screening is defensive and is not the primary trigger condition. Algorithm 1 formalizes the triggering procedure. If the algorithm returns $\perp$, HaReCAP takes no local action and falls back to the original ReCAP leaf LLM.

Therefore, a leaf-reflex rule that corresponds to multiple valid actions is treated as state ambiguity and rejected. When multiple rules hit, the system uses only the support-derived score as a weak ranking signal. If candidate actions differ and the top candidate does not exceed the second candidate by at least 1, the system abstains and returns to ReCAP. On a hit, the unique valid action is used as the next environment action.

3.4 Execution Write-Back and Fallback

After a leaf-reflex hit, only the source of the leaf action changes; ReCAP’s recursive state transition remains unchanged. When the algorithm returns a unique valid action, HaReCAP treats it as a legal grounding result for the current leaf task and immediately interacts with the environment. After the environment returns a new observation, control returns to the original ReCAP parent-node refinement process: the parent node updates the remaining plan, judges subtask progress, and determines the next recursive direction based on the latest feedback. Thus, leaf-reflex triggering does not introduce an independent completion

Algorithm 1 Conservative leaf-reflex triggering**Require:** leaf task τ , valid actions A_t , frozen library $\mathcal{L}$, support gate m , margin $\lambda = 1$ **Ensure:** a valid action $a \in A_t$ or abstention $\perp$

```

1:  $k_\tau \leftarrow \text{canonical\_leaf\_text}(\tau)$ 
2:  $I_\tau \leftarrow \text{ids}(\tau)$ 
3: if  $k_\tau = \emptyset$  or  $I_\tau = \emptyset$  then
4:   return  $\perp$ 
5: end if
6:  $\mathcal{C} \leftarrow \emptyset$ 
7: for all  $s \in \mathcal{L}$  do
8:   if  $k_\tau^s = k_\tau$  and  $n_s \geq m$  then
9:      $a \leftarrow \text{UniqueMatch}(k_a^s, I_\tau, A_t)$ 
10:    if  $a \neq \perp$  then
11:       $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\rho_s, a)\}$ 
12:    end if
13:  end if
14: end for
15: if  $\mathcal{C} = \emptyset$  then
16:   return  $\perp$ 
17: end if
18:  $(\rho^{st}, a^{st}) \leftarrow \text{Top}(\mathcal{C})$ 
19:  $(\rho^{nd}, a^{nd}) \leftarrow \text{SecondTop}(\mathcal{C})$  if it exists
20: if  $a^{nd}$  exists and  $a^{nd} \neq a^{st}$  and  $\rho^{st} - \rho^{nd} < \lambda$  then
21:   return  $\perp$ 
22: end if
23: return  $a^{st}$ 

```

judgment, action repair, or error-recovery strategy. When the triggering algorithm returns $\perp$, the system directly calls the original ReCAP leaf LLM; if the fallback LLM output still cannot be aligned with current valid actions, ReCAP’s original failure handling and backtracking mechanism is used.

4 Experiments

4.1 Experimental Setup

The main experiments use Qwen3.5-72B on Robotouille [2] to evaluate the end-to-end success rate, support-threshold sensitivity, and paired-success reasoning cost of HaReCAP. For Robotouille, the leaf-reflex library is built offline from 121 successful ReCAP baseline trajectories and contains 329 canonical leaf-reflex rules; at runtime, we ablate the minimum execution support threshold over 1, 2, and 4.

Based on this main setting, we further examine whether the mechanism depends on a single task domain or model capability level. The cross-environment experiments are conducted on ALFWorld [15], where a leaf-reflex library is rebuilt from successful ALFWorld trajectories using the same extraction procedure. The cross-model experiments use Qwen3.5-72B and Gemma4-72B to answer

Table 1. End-to-end success rates on Robotouille.

Method	Synchronous	Asynchronous
CoT	6%	0%
ReAct	22%	6%
ReCAP	75%	46%
HaReCAP s=1	76%	50%
HaReCAP s=2	76%	48%
HaReCAP s=4	74%	50%

whether the leaf-reflex mechanism depends on a single model scale or model family. All within-group comparisons use low-temperature decoding, a context token limit of 30000, and a maximum generation length of 2048. Metrics include success rate, LLM calls and token consumption.

We report both all-task and both-success-task results: the former measures end-to-end success over the full task set, while the latter compares execution efficiency only on tasks solved by both ReCAP and HaReCAP. This is important because long-horizon failures are often dominated by high-level planning drift or recovery loops, whose token costs are highly variable; both-success tasks provide more comparable trajectories for evaluating the cost impact of leaf-reflex grounding.

4.2 Robotouille Main Results

Table 1 reports end-to-end success rates on Robotouille with Qwen3.5-27B. CoT and ReAct perform substantially below ReCAP, indicating that this environment requires recursive context management. HaReCAP remains in a similar success-rate range to ReCAP on both the synchronous and asynchronous splits, suggesting that replacing leaf-level grounding does not disrupt ReCAP’s high-level recursive control.

Table 2 further compares the reasoning cost of ReCAP and HaReCAP on Robotouille synchronous/asynchronous both-success tasks under Qwen3.5-27B. Under the condition that the same tasks are completed by both methods, all three support thresholds reduce token consumption relative to ReCAP. Support=2 achieves the largest reduction, lowering tokens by 14.67% and 17.93% on the synchronous and asynchronous splits, respectively. Because the support threshold affects both rule coverage and rule reliability, support=1 covers more cases but includes more low-frequency experience, whereas support=4 is more conservative but reduces the number of triggerable rules. Further increasing the threshold weakens leaf-reflex triggering and makes the system approach the original ReCAP execution mode. Therefore, we use support=2 as the default setting for subsequent cross-environment and cross-model experiments.

These results show that part of ReCAP’s repeated cost in long-horizon recursive control comes from last-mile action grounding. This cost appears not only in relatively deterministic synchronous tasks, but also in asynchronous tasks with

Table 2. Reasoning cost on Robotouille both-success tasks.

Split	Support	Method	Both Succ.	Avg. Tok.	Avg. Calls
Sync	$s = 1$	ReCAP	64	753,455	31.48
		HaReCAP		703,732 ($\downarrow 6.60\%$)	29.19 ($\downarrow 7.30\%$)
	$s = 2$	ReCAP	64	772,053	32.22
		HaReCAP		658,777 ($\downarrow 14.67\%$)	27.66 ($\downarrow 14.15\%$)
	$s = 4$	ReCAP	63	747,628	31.29
		HaReCAP		674,415 ($\downarrow 9.79\%$)	28.25 ($\downarrow 9.72\%$)
Async	$s = 1$	ReCAP	29	1,456,601	56.38
		HaReCAP		1,248,787 ($\downarrow 14.27\%$)	49.28 ($\downarrow 12.60\%$)
	$s = 2$	ReCAP	27	1,357,880	52.85
		HaReCAP		1,114,477 ($\downarrow 17.93\%$)	44.22 ($\downarrow 16.34\%$)
	$s = 4$	ReCAP	29	1,468,649	56.93
		HaReCAP		1,351,911 ($\downarrow 7.95\%$)	53.28 ($\downarrow 6.41\%$)

waiting and interleaved dependencies. It can be replaced by leaf-reflex grounding to reduce inference cost without taking over high-level recursive planning.

4.3 Cross-Environment and Cross-Model Generalization

Robotouille verifies the efficiency gain of HaReCAP within a kitchen-like embodied environment. To separate the portability of the method from task-domain-specific effects, we rebuild a leaf-reflex library on ALFWorld and repeat the evaluation across models. ALFWorld differs from Robotouille in task semantics, interactive objects, and action space, so it tests whether leaf-reflex grounding can still replace repeated leaf action grounding under a new environment interface. Qwen3.5-9B and Gemma4-26B are used to test whether the mechanism depends on Qwen3.5-27B.

Table 3 reports the corresponding end-to-end results. On Qwen3.5-27B, HaReCAP maintains a success rate close to ReCAP and reduces all-task average tokens by 20.7%. On Qwen3.5-9B and Gemma4-26B, HaReCAP also maintains comparable or higher success counts while reducing average tokens and LLM calls.

Table 4 compares both-success tasks across different models on ALFWorld. For all three model settings, HaReCAP reduces average total tokens and LLM calls on tasks solved by both methods. The token reductions are 20.08% for Qwen3.5-27B, 14.21% for Qwen3.5-9B, and 21.09% for Gemma4-26B. Together with the Robotouille paired-success results, this shows that the core gain is not a kitchen-specific action rule, but a general replacement of repeated leaf grounding in ReCAP-style frameworks; the end-to-end behavior of leaf-reflex grounding also does not depend on a single model scale.

Table 3. Cross-environment and cross-model results on ALFWorld.

Model	Method	Acc.	Avg. Tok.	Avg. Calls
Qwen3.5-27B	ReCAP	94.03%	308,624	29.90
	HaReCAP s=2	93.28%	244,697 (↓20.7%)	23.82 (↓20.3%)
Qwen3.5-9B	ReCAP	70.15%	348,457	31.17
	HaReCAP s=2	71.64%	282,035 (↓19.1%)	25.24 (↓19.0%)
Gemma4-26B	ReCAP	66.42%	501,186	43.84
	HaReCAP s=2	75.37%	443,885 (↓11.4%)	36.82 (↓16.0%)

Table 4. Reasoning cost on ALFWorld both-success tasks across models.

Model	Method	Both Succ.	Avg. Tok.	Avg. Calls
Qwen3.5-27B	ReCAP	123	314,589	30.25
	HaReCAP s=2		251,411 (↓20.08%)	24.22 (↓19.94%)
Qwen3.5-9B	ReCAP	83	259,001	27.43
	HaReCAP s=2		222,207 (↓14.21%)	22.65 (↓17.43%)
Gemma4-26B	ReCAP	82	282,698	30.15
	HaReCAP s=2		223,069 (↓21.09%)	24.09 (↓20.11%)

Tables 5 and 6 provide additional cross-model evidence on Robotouille. Table 5 reports end-to-end success rates for Qwen3.5-9B and Gemma4-26B: although 9B performs substantially below 27B overall, HaReCAP does not cause systematic collapse; Gemma4-26B also remains close to ReCAP on both splits. Table 6 compares Gemma4-26B on Robotouille both-success tasks and shows clear reductions in LLM calls and token consumption. The results show that the HaReCAP tends to consume fewer tokens, and this phenomenon has certain generalizability, not being limited to a specific model or dataset.

5 Conclusion

This paper presents HaReCAP, a low-intrusion leaf-level action-grounding method for ReCAP-style recursive LLM agents. It distills frequent leaf decisions from successful trajectories into leaf-reflex rules, and during execution triggers an action only when the rule uniquely matches the current valid-action set; otherwise it falls back to the original ReCAP. Experiments on Robotouille and ALFWorld show that HaReCAP reduces token consumption and LLM calls on commonly successful tasks while maintaining similar end-to-end success rates. The results indicate that part of the runtime cost in ReCAP-style recursive agents comes from repeated leaf action grounding rather than new high-level reasoning. Turning these frequent, verifiable grounding decisions into abstainable leaf-reflex rules

Table 5. Cross-model end-to-end success rates on Robotouille.

Model	Method	Synchronous	Asynchronous
Qwen3.5-9B	ReCAP	32%	5%
Qwen3.5-9B	HaReCAP s=2	44%	10%
Gemma4-26B	ReCAP	53%	26%
Gemma4-26B	HaReCAP s=2	55%	24%

Table 6. Reasoning cost of Gemma4-26B on Robotouille both-success tasks.

Split	Method	Both Succ.	Avg. Tok.	Avg. Calls
Sync	ReCAP	44	909,927	38.66
	HaReCAP s=2		611,890 ($\downarrow 32.75\%$)	26.14 ($\downarrow 32.39\%$)
Async	ReCAP	10	2,440,140	92.60
	HaReCAP s=2		2,000,589 ($\downarrow 18.01\%$)	76.20 ($\downarrow 17.71\%$)

provides a lightweight, auditable, and low-intrusion path for improving the efficiency of long-horizon LLM agents.

References

1. Z. Zhang, T. Chen, W. Xu, A. Pentland, and J. Pei, “ReCAP: Recursive context-aware reasoning and planning for large language model agents,” in *Advances in Neural Information Processing Systems*, 2025.
2. G. Gonzalez-Pumariiega, L. S. Yean, N. Sunkara, and S. Choudhury, “Robotouille: An asynchronous planning benchmark for LLM agents,” in *International Conference on Learning Representations*, 2025.
3. X. Li, N. Yan, and M. Mortazavi, “Embodied task planning via graph-informed action generation with large language model,” arXiv:2601.21841, 2026.
4. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
5. J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, 2022.
6. S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, 2023.
7. M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, T. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al., “Graph of thoughts: Solving elaborate problems with large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 17682–17690, 2024.
8. N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023.

9. M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, 2022.
10. W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al., “Inner monologue: Embodied reasoning through planning with language models,” arXiv:2207.05608, 2022.
11. C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “LLM-Planner: Few-shot grounded planning for embodied agents with large language models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 2998–3009, 2023.
12. G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” arXiv:2305.16291, 2023.
13. J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.
14. C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
15. M. Shridhar, X. Yuan, M.-A. Cote, Y. Bisk, A. Trischler, and M. Hausknecht, “ALFWorld: Aligning text and embodied environments for interactive learning,” in *International Conference on Learning Representations*, 2021.
16. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
17. D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” arXiv:2404.16130, 2024.
18. L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, and H. Xiong, “Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs,” in *Advances in Neural Information Processing Systems*, vol. 37, pp. 37665–37691, 2024.
19. P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
20. M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazare, M. Lomeli, L. Hosseini, and H. Jegou, “The Faiss library,” arXiv:2401.08281, 2024.
21. C. R. Garrett, T. Lozano-Perez, and L. P. Kaelbling, “PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 30, pp. 440–448, 2020.
22. M. Yoo, J. Jang, W.-J. Park, and H. Woo, “Exploratory retrieval-augmented planning for continual embodied instruction following,” in *Advances in Neural Information Processing Systems*, pp. 67034–67060, 2024.

- 23. T. Wang and P. Isola, “Understanding contrastive representation learning through alignment and uniformity on the hypersphere,” in *Proceedings of the International Conference on Machine Learning*, 2020.
- 24. Z. Wu, Z. Wang, X. Xu, J. Lu, and H. Yan, “Embodied task planning with large language models,” arXiv:2307.01848, 2023.